

Predicting Earthquake Building Damage (Nepal 2015) using K-Nearest Neighbors, Neural Network and Random Forest

UE24CS352A Machine Learning - Mini-Project Write-up

Keertika (PES1UG24AM133) and Meruva Pallampati Shashank (PES1UG24AM160)

PES University, Bengaluru | GitHub (private): github.com/keertika19/earthquake-damage-ml

1. Problem Statement

After a major earthquake, engineers must quickly judge how badly thousands of buildings are damaged, but detailed structural analysis of each building is slow. We formulate this as a supervised **3-class classification** problem: given a building's structural and legal attributes, predict its damage grade (1 = low damage, 2 = medium damage, 3 = near-complete destruction). We compare three models: K-Nearest Neighbors (KNN), a Neural Network (NN) and a Random Forest (RF).

2. Dataset

We use the *Richter's Predictor: Modeling Earthquake Damage* data (DrivenData), built from the 2015 Gorkha (Nepal) earthquake survey. It has **260,601 labelled buildings** with **38 features**: geographic region IDs, number of floors, age, area and height percentages, foundation, roof and floor types, superstructure materials, legal ownership, number of families and secondary-use flags (8 features are categorical, the rest numeric or binary). The classes are imbalanced: roughly 10% grade 1, 57% grade 2 and 33% grade 3. The competition test file (86,868 buildings) has no public labels, so we hold out our own test data: a **stratified 80/20 split** (random seed 42) gives 208,480 training and 52,121 validation buildings that the models never see during training.

3. Approach

Preprocessing. Categorical features are one-hot encoded. For KNN and the NN, numeric features are standardised (zero mean, unit variance) because both are sensitive to feature scale; the Random Forest uses unscaled features. **Metrics.** Micro-averaged F1 (equal to accuracy for single-label multiclass data), macro-averaged F1 (which weights the three classes equally and so exposes the imbalance), per-class precision/recall and confusion matrices.

- **KNN:** after scaling, the 20 best features are kept with SelectKBest (ANOVA F-test); $k = 7$ neighbours with Euclidean distance.
- **Neural network:** a two-layer MLP (one hidden layer of 64 ReLU units, softmax output), L2 regularisation ($1e-4$), mini-batches of 1000, Adam optimiser (learning rate $1e-3$) and early stopping on a held-out part of the training data.
- **Random Forest:** grid search with 3-fold cross-validation over $n_estimators \{100, 200\}$ and $min_samples_leaf \{1, 5\}$, selecting by micro-F1, then refitting the best setting on the full training split.

4. Implementation Overview

The project is written in Python with scikit-learn, pandas and matplotlib, and was run on Google Colab. **train.py** loads the CSVs, creates the stratified split, builds one scikit-learn *Pipeline* per model (preprocessing + classifier), fits and evaluates it, and saves metrics (*results/metrics.csv*), confusion matrices, an NN training curve, RF feature importances and the trained models. **demo.py** reloads a saved model and predicts damage for random buildings from the held-out validation split for the live demonstration. The README explains setup and usage; the dataset and saved models are not committed.

5. Results (held-out validation set, 52,121 buildings)

Model	Train micro-F1	Val micro-F1	Val macro-F1	Fit time	Predict time
K-Nearest Neighbors	0.673	0.646	0.573	0.5 s	46.4 s
Neural Network (MLP)	0.675	0.668	0.594	56.9 s	0.2 s
Random Forest	0.987	0.716	0.657	380.2 s	2.6 s

Per-class results of the best model (Random Forest):

Damage grade	Precision	Recall	F1-score	Support
1 (low)	0.643	0.479	0.549	5,025
2 (medium)	0.723	0.824	0.770	29,652
3 (destruction)	0.718	0.600	0.654	17,444

6. Conclusions

- **Random Forest performed best** (validation micro-F1 0.716, macro-F1 0.657), followed by the Neural Network (0.668) and KNN (0.646).
- **Overfitting vs underfitting:** the RF reaches 0.987 on training data but 0.716 on validation, a clear overfitting gap. KNN and the NN score almost the same on both sets (about 0.65-0.67), which suggests underfitting: they are too simple, or lose information (KNN uses only 20 features).
- **Class imbalance:** all models favour the majority class (grade 2). Even the RF recalls only 48% of grade-1 and 60% of grade-3 buildings, which is why macro-F1 is lower than micro-F1.
- **Efficiency:** KNN trains instantly but is the slowest to predict (46 s); the NN predicts fastest; the RF is the slowest to train because of the grid search.
- **Future work:** class weighting or resampling, limiting tree depth, a larger or deeper network, gradient-boosted trees, and submitting predictions to the DrivenData leaderboard for an independent test score.

Note: validation scores were also used to compare the models, so they are slightly optimistic compared with a truly unseen test set.